

Reviewer Pack — Singular Value Gap in Read-Twice BPs for IP_2

Entry f3a21c12c399 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 21:00:26 UTC

Singular Value Gap in Read-Twice BPs for IP_2

Entry ID: f3a21c12c399 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 21:00:26 UTC

1. Conjecture

Field A (mathematical branch): Random Matrix Theory **Field B** (complexity object): Read-Twice Branching Programs for IP_2

Statement:

The sum of singular values of the transition matrix of a read-twice BP for IP_2 is $\Omega(n)$, whereas for read-once BPs, it is $O(\log n)$.

Rationale (proposer’s reasoning):

Random matrix theory provides tools to analyze the spectral structure of BP transition matrices. IP_2’s inherent symmetry forces large singular values in read-twice BPs, contrasting with the logarithmic bounds for read-once BPs, which could reflect structural differences in their computation.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2e27e4359e7862f7

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_transition_matrix(n):
        T = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    T[i][j] = random.randint(1, 10)
                    T[j][i] = T[i][j]
        return T

    def sum_of_singular_values(T):
        n = len(T)
        U = [[0] * n for _ in range(n)]
        S = [0] * n
        V = [[0] * n for _ in range(n)]

        # Compute the transition matrix for read-twice BPs (IP_2)
        T_twice = generate_transition_matrix(n)

        # Compute singular values using power iteration method
        def power_iteration(A, max_iter=100):
            v = [random.random() for _ in range(n)]
            v /= math.sqrt(sum(x**2 for x in v))
            for _ in range(max_iter):
                v = A @ v
                v /= math.sqrt(sum(x**2 for x in v))
            return v

        def svd(A):
            U = [[0] * n for _ in range(n)]
            S = [0] * n
            V = [[0] * n for _ in range(n)]

            for i in range(n):
                v = power_iteration(A)
                u = A @ v
                s = math.sqrt(sum(x**2 for x in u))
                U[i] = u / s
                S[i] = s
                V[i] = v
```

```

        return U, S, V

    U, S, V = svd(T_twice)
    sv_sum_twice = sum(S)

    return sv_sum_twice

n = random.randint(5, 40)
sv_sum_twice = sum_of_singular_values(n)

return {
    "metric_name": "sum_of_singular_values",
    "metric_value": sv_sum_twice,
    "instances_tested": 1,
    "conjecture_holds": True if sv_sum_twice >= n else False,
    "counterexample": "" if sv_sum_twice >= n else f"n={n}, sum_of_singular_values={sv_sum_twice}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={first_failing_seed}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fdb702f.py", line 85, in <module>

```

    result = run_trial(seed)
    ^^^^^^^^^^^^^^^^^

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fdb702f.py", line 69, in run_trial

```

    sv_sum_twice = sum_of_singular_values(n)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6fdb702f.py", line 31, in sum_of_singular
    n = len(T)
    ^^^^^^
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with TypeError due to invalid len() call on integer | next: Debug the sum_of_singular_values function to fix type handling

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	127639
2	preregistration	ollama_remote	qwen3:8b	0	0	32692
3	novelty	ollama_remote	qwen3:8b	0	0	28277
4	novelty	ollama_remote	qwen3:8b	0	0	20725
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	23220
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13346
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10365
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10009
9	judge	ollama_remote	qwen3:8b	0	0	19784

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 286057 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/f3a21c12c399.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/f3a21c12c399.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/f3a21c12c399.tar.gz (if generated)